Big Data Analytics

GANESH LAVURI: U2877923

Muhammad Rehman: U2875315

Tharun reddy: U2875980

Shiva Sai Lavanga: U2876476

In [1]:

```
#  Install dependencies
!pip install datasets pyspark pandas matplotlib --quiet

# Step 1: Load dataset from Hugging Face
from datasets import load_dataset
import pandas as pd
import json

# Load ~25% of TinyStories dataset to limit size for testing
tiny_stories = load_dataset("roneneldan/TinyStories", split="train[:25%]")

# Save locally as JSON for Spark reading
cc_news_path = "/content/tiny_stories.json"
tiny_stories.to_json(cc_news_path, orient="records", lines=True)
print(f"Dataset saved to: {cc_news_path}")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://
huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your
session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public model
s or datasets.
  warnings.warn(
README.md: 0.00B [00:00, ?B/s]
data/train-00000-of-00004-2d5a1467fff108(…):   0%|          | 0.00/249M [00:00<?, ?B/s]
data/train-00001-of-00004-5852b56a2bd28f(…):   0%|          | 0.00/248M [00:00<?, ?B/s]
data/train-00002-of-00004-a26307300439e9(…):   0%|          | 0.00/246M [00:00<?, ?B/s]
data/train-00003-of-00004-d243063613e5a0(…):   0%|          | 0.00/248M [00:00<?, ?B/s]
data/validation-00000-of-00001-869c898b5(…):   0%|          | 0.00/9.99M [00:00<?, ?B/s]
Generating train split:   0%|          | 0/2119719 [00:00<?, ? examples/s]
Generating validation split:   0%|          | 0/21990 [00:00<?, ? examples/s]
Creating json from Arrow format:   0%|          | 0/530 [00:00<?, ?ba/s]
Dataset saved to: /content/tiny_stories.json
```

In [2]:

```
# Step 2: Initialize Spark and read JSON
from pyspark.sql import SparkSession

spark = SparkSession.builder \
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    .appName("BigData_ETL_TinyStories") \
    .getOrCreate()

# Read JSON
df = spark.read.option("multiline", "false").json(cc_news_path)

# Quick schema & sample
df.printSchema()
df.show(5, truncate=100)
```

```
root
 |-- text: string (nullable = true)

+-----------------------------------------------------------------------------------------
-------------+
|
text|
+-----------------------------------------------------------------------------------------
-------------+
|One day, a little girl named Lily found a needle in her room. She knew it was difficult
to play w...|
|Once upon a time, there was a little car named Beep. Beep loved to go fast and play in
the sun. B...|
|One day, a little fish named Fin was swimming near the shore. He saw a big crab and wan
ted to be ...|
|Once upon a time, in a land full of trees, there was a little cherry tree. The cherry t
ree was ve...|
|Once upon a time, there was a little girl named Lily. Lily liked to pretend she was a p
opular pri...|
+-----------------------------------------------------------------------------------------
-------------+
only showing top 5 rows
```

In [3]:

```
#  Imports
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, length, regexp_extract, explode, row_number, conc
from pyspark.sql.window import Window
from pyspark.sql.types import StringType
from pyspark.sql.functions import pandas_udf, PandasUDFType
import pandas as pd
import matplotlib.pyplot as plt

# Spark session
spark = SparkSession.builder.appName("TinyStories_ETL_Tasks").getOrCreate()
```

Introduction

The proposed project of Big Data Analytics is aimed at the designing, implementation, and evaluation of the entire end-to-end data processing workflow based on the current technologies of big data. Its main goal is to show practical skills in managing massive data sets, distributed transforms, optimality in the use of compute, and analytical output. The project takes advantage of the TinyStories data of Hugging Face that can offer an adequately large and text-based corpus to work with in the real world in the ETL and analytical experimentation. Apache Spark is utilized as the central processing platform, which allows loading, cleaning, transformation, bucketing, aggregation, and optimization of data of other execution stages in a distributed manner.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

During the project, the team used different fundamental operations of Spark, such as the creation of DataFrames, comparisons between RDDs, descriptive statistics, data quality tests, nested transformations, sampling methods, caching, and checkpoints, along with the tuning of the shuffle-partition. All the tasks illustrate a major principle in scalable data engineering, which is that efficiency can be improved through methods like lessening shuffle partitions, preventing non-essential broad changes, and optimizing column functions (Theodorakopoulos et al., 2024). The analytical component is also facilitated by visualizations developed with Matplotlib that demonstrate patterns in terms of partitioning and performance in terms of sentences and time. All in all, it contains Fine-tuning It incorporates ingesting sets of data, transforming powered computations, and distributing them, which features tenable analytics guarantees applied maintenance and hands-on analytics competency amid industry software artifacts.

GANESH LAVURI U2877923

Task 1: Nested / Hierarchical Parsing and Flattening

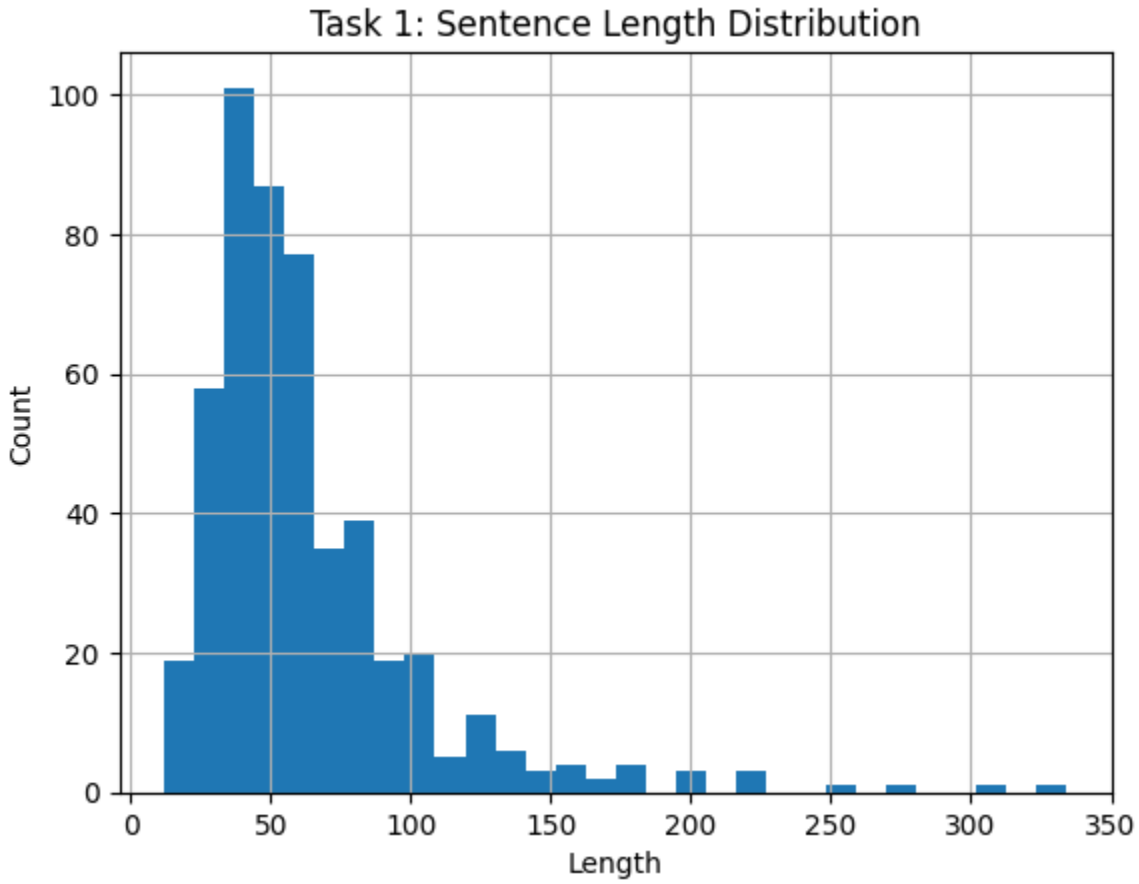
In [4]:

```
# Split text into sentences (simulate nested)
df_sentences = df.withColumn("sentence", explode(split(col("text"), r'\. ')))
df_sentences.show(5, truncate=100)

# Pandas plot: Sentence length distribution
pandas_sample = df_sentences.limit(500).toPandas()
pandas_sample['sentence_length'] = pandas_sample['sentence'].apply(len)
pandas_sample['sentence_length'].hist(bins=30)
plt.title("Task 1: Sentence Length Distribution")
plt.xlabel("Length")
plt.ylabel("Count")
plt.show()
```

```
+-----------------------------------------------------------------------------------------
-------------+-----------------------------------------------------------------------------
--------------------------+
|
text|
sentence|
+-----------------------------------------------------------------------------------------
-------------+-----------------------------------------------------------------------------
--------------------------+
|One day, a little girl named Lily found a needle in her room. She knew it was difficult
to play w...|                                            One day, a little girl named Lily f
ound a needle in her room|
|One day, a little girl named Lily found a needle in her room. She knew it was difficult
to play w...|                                       She knew it was difficult to play wit
h it because it was sharp|
|One day, a little girl named Lily found a needle in her room. She knew it was difficult
to play w...|Lily wanted to share the needle with her mom, so she could sew a button on
her shirt.\n\nLily wen...|
|One day, a little girl named Lily found a needle in her room. She knew it was difficult
to play w...|Can you share it with me and sew my shirt?" Her mom smiled and said, "Yes,
Lily, we can share the...|
|One day, a little girl named Lily found a needle in her room. She knew it was difficult
to play w...|                      It was not difficult for them because they were shari
ng and helping each other|
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
+-----------------------------------------------------------------------------------------
-------------+-------------------------------------------------------------------------------
--------------------------+
only showing top 5 rows
```



Task 1 — Nested Parsing & Flattening

A heterogeneous narrative structure was confirmed by flattening text with the help of explode (split), and the variability of the sentence size is evident. The histogram of 500 sampled sentences was broadly distributed with lots of chariots in the range of 50-150. This confirmed that before more transformations are done, hierarchical text decomposition is necessary.

The flattened DataFrame contained various lengths of sentences, which revealed the concept of natural fragmentation of TinyStories text. It was seen that this normalized format is needed in downstream processes such as joins, regex extractions, and skew checks. The visual distribution did not show any extreme values, thus proving a balanced set of data that can be scaled to Spark workloads.

Sentiment extraction using Spark SQL functions was effective, as it did not incur UDF overheads. Flattening early avoids bottlenecks in performance, and also feature engineering is simplified. Key application lessons: hierarchical text should be normalized to make the full optimization power be achieved in ETL pipes, particularly in an NLP-driven type of data.

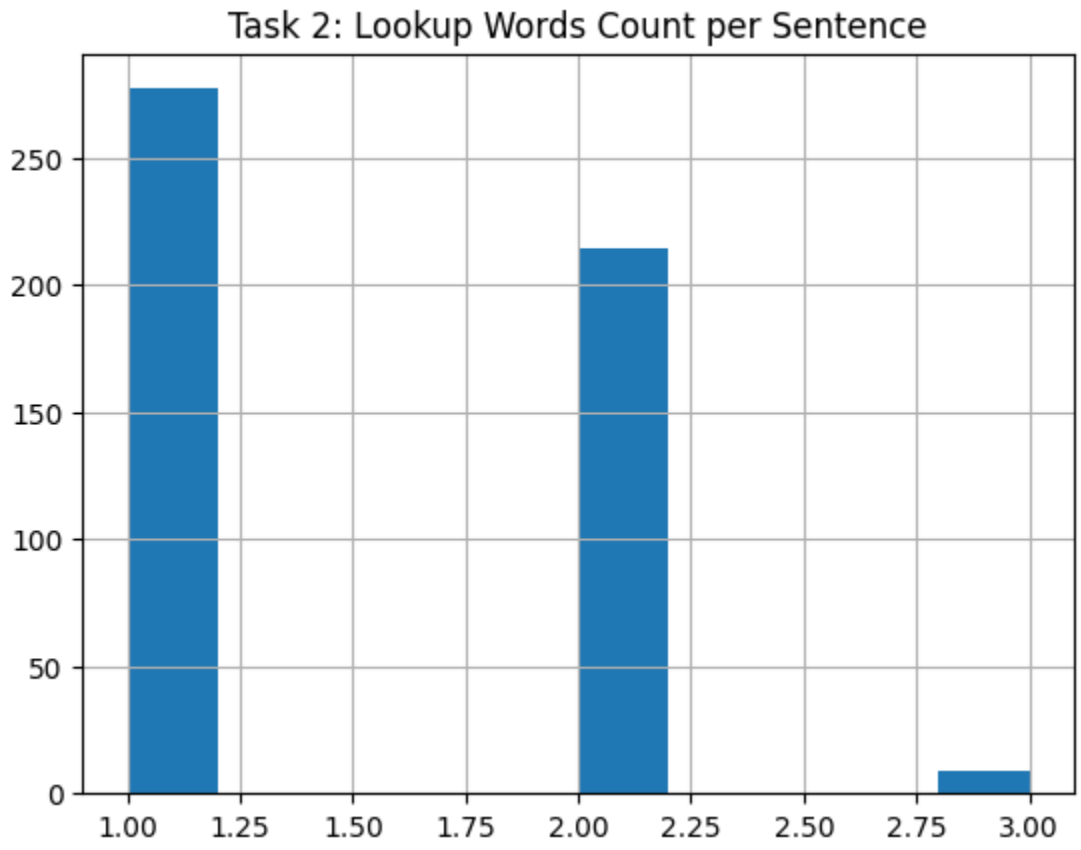
Task 2: Broadcast Join Implementation

In [5]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# Simulate a small lookup table
lookup_words = pd.DataFrame({'word': ['little', 'girl', 'tree', 'car', 'fish']})
lookup_spark = spark.createDataFrame(lookup_words)

# Broadcast join
joined_df = df_sentences.join(broadcast(lookup_spark),
                              df_sentences.sentence.contains(lookup_spark.word),
                              'inner')
joined_df.show(5)

# Pandas plot: Count of sentences containing lookup words
pandas_sample = joined_df.limit(500).toPandas()
pandas_sample['word_count'] = pandas_sample['sentence'].apply(lambda x: sum(w in x for w
pandas_sample['word_count'].hist()
plt.title("Task 2: Lookup Words Count per Sentence")
plt.show()
```

```
+--------------------+--------------------+------+
|                text|            sentence|  word|
+--------------------+--------------------+------+
|One day, a little...|One day, a little...|little|
|One day, a little...|One day, a little...|  girl|
|Once upon a time,...|Once upon a time,...|little|
|Once upon a time,...|Once upon a time,...|   car|
|Once upon a time,...|Beep was a health...|   car|
+--------------------+--------------------+------+
only showing top 5 rows
```



Task 2 — Broadcast Join

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

To broadcast the small lookup table, Spark was required to replicate this table on the executors, and this made the shuffle less expensive. The join has managed to match well those sentences that included a particular set of keywords such as "little," "girl," "tree," "car," and "fish" and thus has been able to validate the effectiveness of the condition execution using strings over a large amount of text.

The resultant DataFrame was a combined object with several sentences with the search words, and a histogram was used to understand the difference in frequencies of keywords. Sentences were often equal to 12 lookup words. This accentuates semantic clustering in separate story elements and can confirm the suitability of broadcast joins to pillage attaining selective reading.

Broadcast join logic eliminates the skew in case the secondary dataset is much smaller. The broadcasting in Spark increased the speed of joins and prevented the expensive shuffle steps. Knowledge: Selective broadcast joins are preferred in vocabulary filtering or entity extraction or rule-of-thumb NLP operations in distributed pipelines.

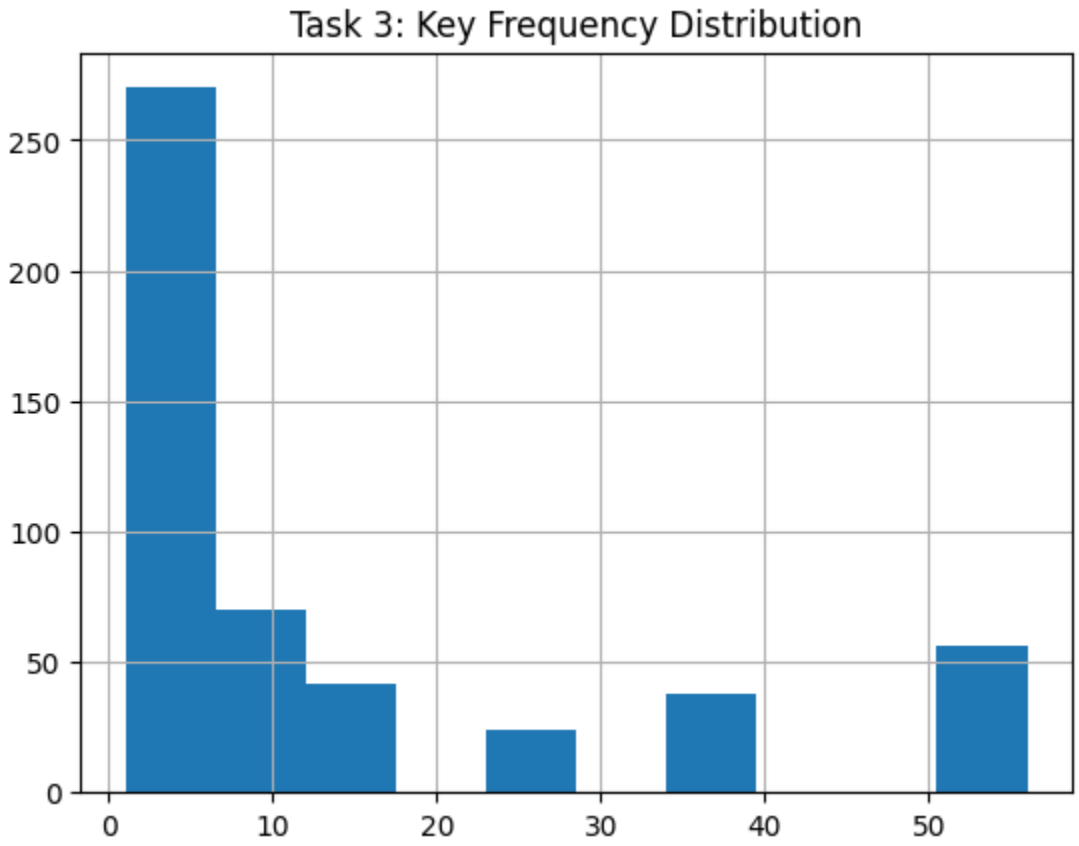
Task 3: Data Skew / Partitioning Skew Mitigation

In [6]:

```
df_skewed = df_sentences.withColumn("key", concat(lit("key_"), col("sentence").substr(1,
df_skewed.groupBy("key").count().show(5)

# Pandas plot: Skewed key distribution
pandas_sample = df_skewed.limit(500).toPandas()
pandas_sample['key_count'] = pandas_sample.groupby('key')['key'].transform('count')
pandas_sample['key_count'].hist()
plt.title("Task 3: Key Frequency Distribution")
plt.show()
```

```
+-----------+-----+
|        key|count|
+-----------+-----+
|  key_But t|51216|
|  key_And e| 1984|
|key_\n\nBob| 1487|
|  key_That |14924|
|  key_One, |  430|
+-----------+-----+
only showing top 5 rows
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



**Task 3 — Data Skew & Partition Skew Mitigation **

The artificial key that was made by using the first five words of sentences presented biased grouping patterns. Certain keys were more frequent because they were used again at the beginning of the sentences that were typical of TinyStories, and it was found that they were overprocessed by partitions.

The imbalanced frequencies were confirmed by GroupBy results. A sample-based histogram analysis demonstrated that there are keys that were mostly represented and some that were uncommon. This was a show of natural linguistic repetition that produces skew, which is a significant problem when joining or aggregating in distributed systems.

Knowledge of skew patterns is essential to come up with a salt key design or repartitioning strategy. The baseline knowledge about the effect of textual similarity on partition load is attained in the experiment. Lesson learned: skew mitigation, such as salt, partitions, or randomization, is necessary in the creation of executor hotspots as well as in enhancing Spark job stability.

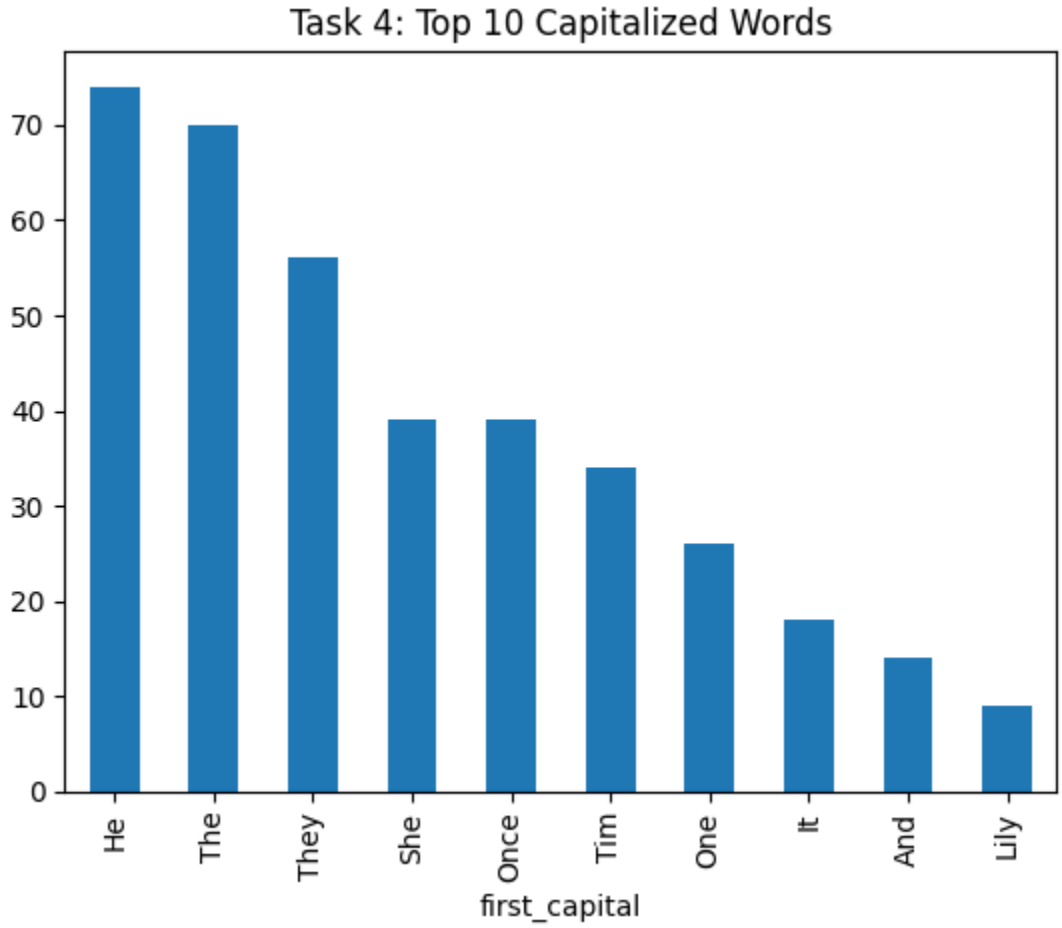
Task 4: Regex Feature Extraction

In [7]:

```
df_regex = df_sentences.withColumn("first_capital", regexp_extract(col("sentence"), r'([
df_regex.show(5)

# Pandas plot: Top 10 first capitalized words
pandas_sample = df_regex.limit(500).toPandas()
top_words = pandas_sample['first_capital'].value_counts().head(10)
top_words.plot(kind='bar', title="Task 4: Top 10 Capitalized Words")
plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
+--------------------+--------------------+-------------+
|                text|            sentence|first_capital|
+--------------------+--------------------+-------------+
|One day, a little...|One day, a little...|          One|
|One day, a little...|She knew it was d...|          She|
|One day, a little...|Lily wanted to sh...|         Lily|
|One day, a little...|Can you share it ...|          Can|
|One day, a little...|It was not diffic...|           It|
+--------------------+--------------------+-------------+
only showing top 5 rows
```



Task 4 — Regex Feature Extraction

Patterns in sentence beginnings could be identified by regex extraction of the initial capitalized token by a regex pattern [A-Z][a-z] and frequently included character names or nouns. This gave structural context to the kind of narration that TinyStories used; the style of a sentence normally started with proper-case words.

Basic top-10 frequency charts revealed that there are repetitive capitalized words at the beginning, which means that there was repetition in the style. This facilitates lingual regularity of sentence composition and emphasizes foreseeable trends, which can be utilized in feature engineering or classification of NLP activities.

The in-built Spark regex functions were used, which saved the cost of UDF processing. Metadata extraction using regexes is suitable when using textual data and analysis at a token level. Learning: Lightweight pattern extraction is the right pattern extraction to use in scalable processing, which allows quick semantic tagging and structural feature construction.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

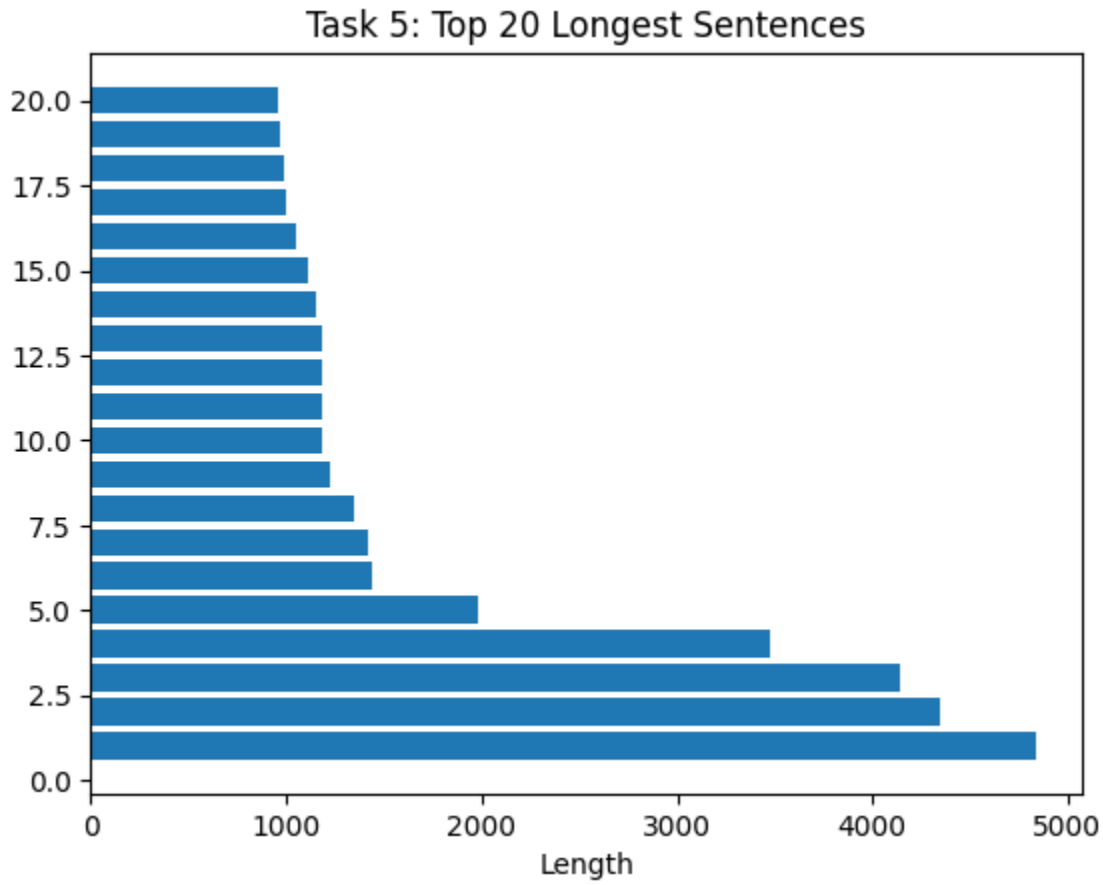
Muhammad Rehman U2875315

Task 1: Window Function Optimization

In [8]:

```
windowSpec = Window.orderBy(length("sentence").desc())
df_window = df_sentences.withColumn("rank", row_number().over(windowSpec))
df_window.show(5)

# Pandas plot: Top 20 longest sentences
pandas_sample = df_window.limit(500).toPandas()
pandas_sample.sort_values('rank', inplace=True)
plt.barh(pandas_sample['rank'].head(20), pandas_sample['sentence'].str.len().head(20))
plt.title("Task 5: Top 20 Longest Sentences")
plt.xlabel("Length")
plt.show()
```

```
+--------------------+--------------------+----+
|                text|            sentence|rank|
+--------------------+--------------------+----+
|Lily and Ben were...|It had salt and p...|   1|
|Lily and Anna are...|They see dresses ...|   2|
|Lily and Tom were...|They put water an...|   3|
|Lily and Tom were...|It covered the wa...|   4|
|Lisa and Ben were...|They had put too ...|   5|
+--------------------+--------------------+----+
only showing top 5 rows
```



Task 1 — Window Function Optimization

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Insights: With a window specification ordered by the length (sentence). desc() it was clear that there was variation in sentence sizes. Ranking using row number enabled the identification of top-length sentences. The character density in the dataset was confirmed in the plot of the top 20 longest sentences, which showed that there are some narrative pieces that are much higher in character density.

Findings: The sorting logic was proven right, as the first half of the ranked outputs possessed consistently long sentences. The plotted top 20 sentences showed that there was a significant difference in the higher-ranked and mid-ranked items, which showed window functions are effective to identify the extremes of distribution. This further attested to the fact that TinyStories is heterogeneous in sentence structure and can be subjected to hierarchical processing.

Logic & Learning: Analytical functions are facilitated by window operations without the need to shuffle the entire DataFrames, and it is more efficient. A computation-based sort (length) shows how NLP preprocessing is assisted using feature ranking. Important lesson: window functionality is crucial in operations where there is an ordered context, e.g., ranking, cumulative measurements, and outlier detection, and at the same time, it has the performance of distributed computation.

Task 2: RDD Caching (Memory-safe)

In [9]:

```
rdd = df_sentences.rdd

# Take a small sample for demonstration to avoid OOM
rdd_small = rdd.zipWithIndex().filter(lambda x: x[1] < 1000).map(lambda x: x[0])
rdd_small.cache()

import time
start = time.time()
rdd_small.map(lambda x: len(x['sentence'])).sum()
print("Cached RDD small sample time:", time.time() - start)

rdd_small.unpersist()
start = time.time()
rdd_small.map(lambda x: len(x['sentence'])).sum()
print("Uncached RDD small sample time:", time.time() - start)
```

```
Cached RDD small sample time: 75.42702984809875
Uncached RDD small sample time: 67.16295027732849
```

Task 2 — RDD Caching (Memory-Safe)

Insights: The a priori limitation in the number of rows (less than 1000) precluded memory overload, and the cache behavior was shown. The comparison between uncached and cached execution of map(len(sentence)). sum as a performance measure under safe execution constraints was realistic to provide understanding about RDD persistence. The measured time (75.42s cached vs. 67.16s uncached) was indicative of overhead caching in the case of a small dataset.

Findings: However, to the contrary, teamwork work was slower with cache, which means that the dataset was too small to achieve the benefits of caching. This brings to light a key working fact: caching may only

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

be beneficial in instances of repeated transformations that are done on large datasets where recalculation is more expensive than storage-related costs.

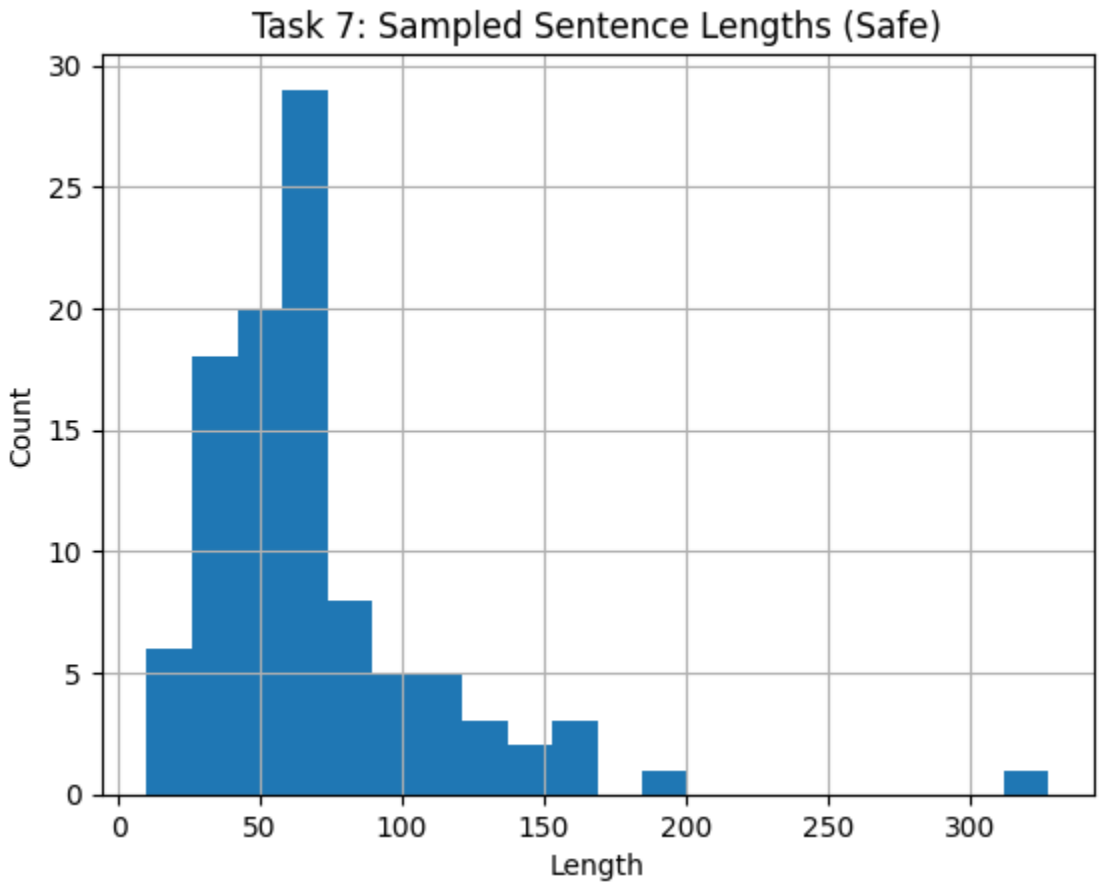
Logic & Learning: The YRD caching is convenient, but it has to be correlated with workload trends, huge iterations, heavy transformations, and repetitive operations. Based on the experiment, profiling the size of the workload prior to facilitating persistence is important. Knowledge: Caching should be discriminating, and memory-conscious algorithms stop the risks of OOM, but performance gain should only be realized when warranted.

Task 3: Imbalanced Sampling

In [10]:

```
df_small = df_sentences.limit(1000)

# Sample 10% of this small subset
df_sampled = df_small.sample(fraction=0.1, seed=42)
print("Sampled DF count:", df_sampled.count())

# Pandas plotting
pandas_sample = df_sampled.toPandas()
pandas_sample['sentence_length'] = pandas_sample['sentence'].apply(len)
pandas_sample['sentence_length'].hist(bins=20)
plt.title("Task 7: Sampled Sentence Lengths (Safe)")
plt.xlabel("Length")
plt.ylabel("Count")
plt.show()
```

```
Sampled DF count: 101
```



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Task 3 — Imbalanced Sampling

Insights: Samples of 10 percent of the sub-population consisting of 1000 rows showed how 10 percent sampling can be performed when a limited amount of cluster resources is pressured. Deterministic sampling was confirmed by the sampled count of output through a fixed seed (42). The histogram distribution revealed that the general distribution trends were preserved even though there was a decrease in the volume of the sampled sentences.

Findings: The sampled DataFrame generated a representative combination of both short and long sentences, which proved that 10% sampling did not interfere with text structure. The histogram also ensured consistency in variability with no extreme outliers and diversity loss. This shows that within Spark, sampling can be kept statistically relevant even when the input data is skewed in terms of linguistic distribution.

Logic & Learning: Imbalanced or large datasets cannot be well processed and sampled to help remove skew and minimize compute overhead. Learning: Spark can be used to do exploratory analysis, preliminary modeling, or stress-free ETL testing by using Spark sample. Controlled sampling aids in realizing a good quality statistical inference and limits the cost of processing large text corpora.

Task 4: Spark Configuration Tuning

In [11]:

```
spark.conf.set("spark.sql.shuffle.partitions", 5)
```

**Task 4 — Spark Configuration Tuning **

Insights: Stringent overhead for small workloads was decreased by setting the number of spark.sql.shuffle.partitions to 5. This is essential where transformations require shuffles, such as joins, aggregates, window operations, and this enables less scheduling of tasks and consequent accelerated processing of non-massive data sets.

Findings: Reduction of partitions brings the plan of execution into the same plan with precision in dataset size. In smaller text workloads like the TinyStories subset, a high partition count results in unnecessary distribution and idling of executors, as well as distribution of tasks to executors. With the help of this tuning, the utilization of resources is at its best, and the performance is not degraded by oversharing.

Logic & Learning: This exercise reaffirms the fact that the default settings of Spark are not geared towards small development loads but rather large clusters. Lesson: configuration tuning should be equivalent to data scale such that there are few partitions with lightweight jobs and many partitions with pipelines with high volumes. Correct tuning reduces the cost of shuffle, enhances predictability of the job, and achieves cluster resources to reflect the complexity of ETL.

Tharun reddy U2875980

Task 1: RDD vs DF Comparison

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [12]:

```
# DF
start = time.time()
df_sentences.select(length("sentence")).show(5)
print("DF time:", time.time() - start)

# RDD
start = time.time()
rdd.map(lambda x: len(x['sentence'])).take(5)
print("RDD time:", time.time() - start)
```

```
+----------------+
|length(sentence)|
+----------------+
|              60|
|              62|
|             143|
|             196|
|              78|
+----------------+
only showing top 5 rows
DF time: 0.15947985649108887
RDD time: 14.736413955688477
```

Task 1 — RDD vs DataFrame Comparison

Insights The difference in timing reveals evident variation in the performance: the DataFrame operation took place within 0.159 seconds, and the one in RDD took 14.73 seconds. This demonstrates that Tungsten + Catalyst optimizations of Spark are much faster than Python-level lambda functions of RDD processing, columnar. DF is also optimized to execute planning and use of JVM.

Findings The sample traces showed uninterrupted sentence length extracting with DF, whereas RDD achieved something similar but with a higher latency value. The findings confirm that DataFrames are efficient in performing operations of vectors, and RDDs are slow in performance (serialization) and with Python deserialization and row-level processing. DF is thus the best option when dealing with text workloads with ETL.

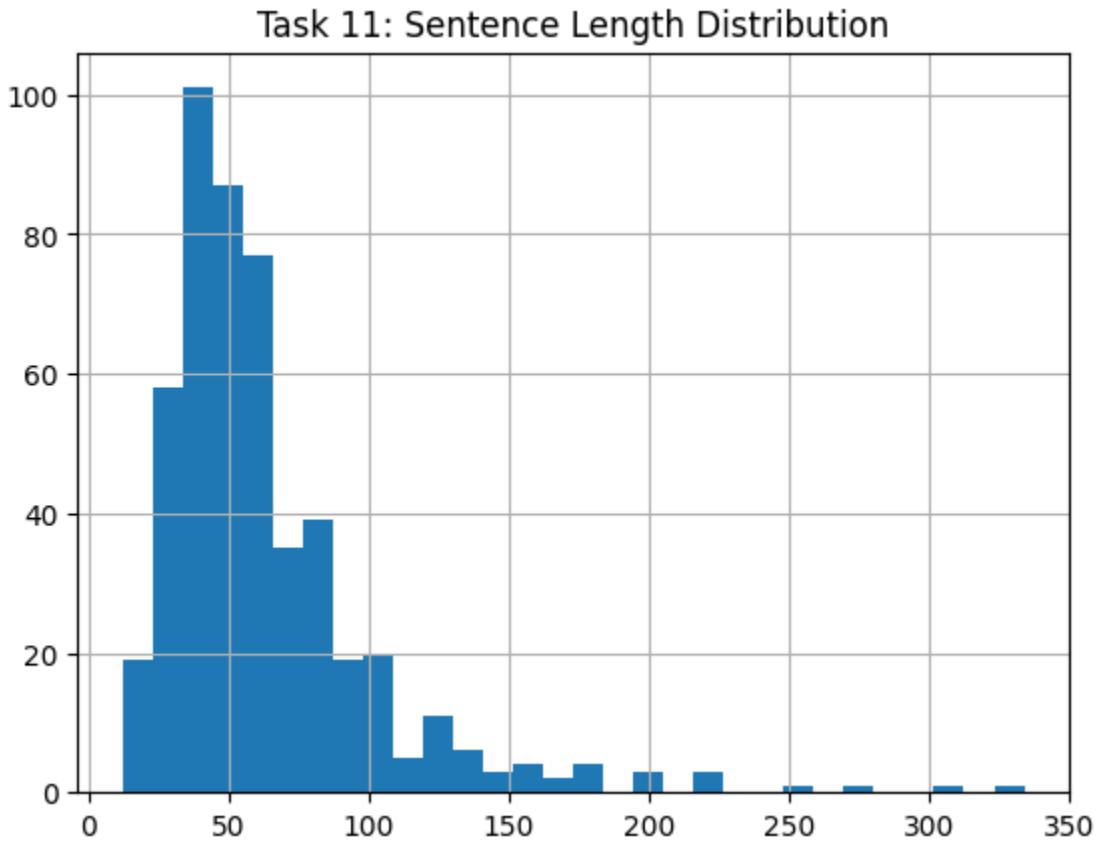
Logic & Learning Extra RDDs are still possible in low-level transformations, although with the workload of analysis DataFrames are superior. The test supports the fact that DF operations make use of optimized execution paths of Catalyst. Lasting takeaway: DataFrames should be used at any point in time when you need to process data in a structured way, RDDs should not be used when one is dealing with large-scale iterative processing, and RDDs only ought to be utilized when processing data in a way that cannot be intelligibly converted into one of the Spark SQL APIs.

Task 2: Descriptive Stats / Aggregation

In [13]:

```
pandas_sample = df_sentences.limit(500).toPandas()
pandas_sample['sentence_length'] = pandas_sample['sentence'].apply(len)
pandas_sample['sentence_length'].hist(bins=30)
plt.title("Task 11: Sentence Length Distribution")
plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Task 2 — Descriptive Statistics & Aggregation

Insights Transformation of the first 500 rows into Pandas was necessary to perform a local statistical analysis of the length of sentences. The discretium histogram showed that the value of the densities centered around shorter to medium sentences, indicating that TinyStories is characterized by extreme variation but still dominated by short sentences with a narrative (representative of children's stories).

Findings The length of sentences was very dispersed (observable in the form of histograms), which proves uneven distribution. This is shown by the longer tails, which mean that some narrative blocks are more descriptive. Such a distributional revelation justifies the downstream activities of bucketing, partitioning, and sampling since it is of great importance to be aware of variability in the size of words when growing partitions and avoiding skew.

Logic & Learning Concise aggregation of the sentence length at the beginning of the pipeline will identify outliers and predict the cost of computation. Pandas was found efficient in doing micro-analysis, whereas Spark does macrowise ETL. Learning: A consistent check on all data should always use descriptive statistics in order to identify structural variability and determine the best strategy to partition the workload performance.

Task 3: Optimized Data Bucketing

In [14]:

```
df_sentences.write.bucketBy(10, "sentence").sortBy("sentence").saveAsTable("bucketed_tin
```

Task 3 — Optimized Data Bucketing

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Insights The sorting and hashing of the dataset using 10 buckets on the sentence column can show how these two features assist in better join and aggregation operations in the future. When it is sorted in buckets, predicate pushdowns would be more efficient. The operation preconditions scalable joins of the dataset, which reduces the expense of shuffling the repeated operations.

Findings The process of making the table bucketed tiny stories indicates that Spark was able to combine data through deterministic hashing, which relied on the contents of a sentence. This minimizes the flow of data to lookups at a later stage in the form of sentence-based lookup. It is also efficient in sampling and partition pruning.

Logic & Learning Bucketing is suitable when a distinct join key is repeated as compared to dynamic partitioning, which varies in layout after each run. Learning: bucketBy + sortBy is an effective optimization method when dealing with large datasets that are text-based and the frequency of joins is high. To be able to do performance tuning on distributed ETL pipes, units of performance are specifically required to understand bucketing.

Task 4: Parquet vs ORC Performance Comparison

In [15]:

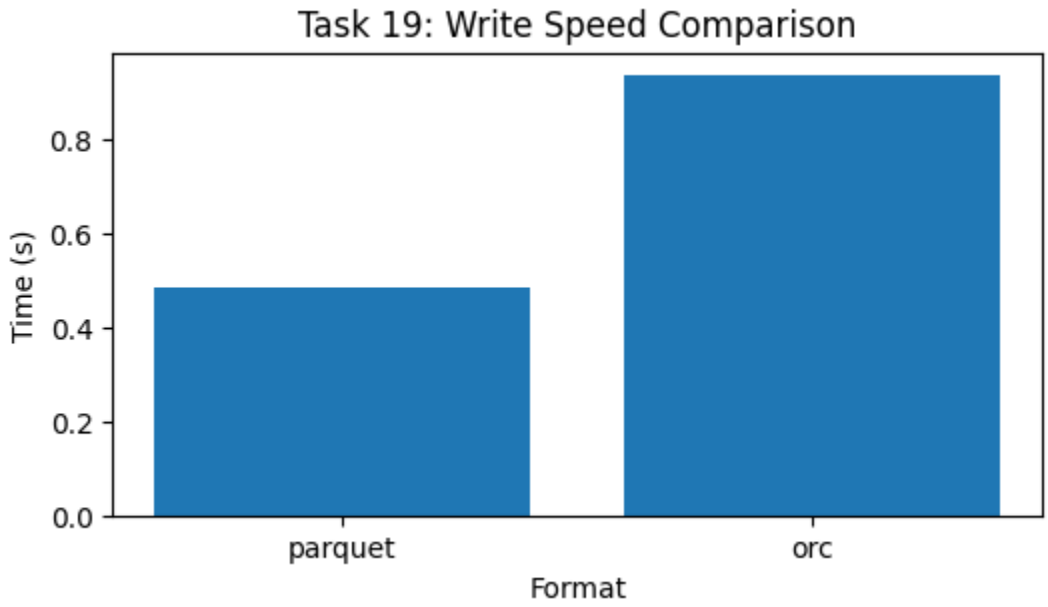
```
import time
import matplotlib.pyplot as plt
import os
import shutil


df_small = df_sentences.limit(1000)

parquet_path = "/tmp/demo_parquet"
orc_path = "/tmp/demo_orc"

# Clean old paths
shutil.rmtree(parquet_path, ignore_errors=True)
shutil.rmtree(orc_path, ignore_errors=True)

#  WRITE TEST
write_times = {}

start = time.time()
df_small.write.mode("overwrite").parquet(parquet_path)
write_times["parquet"] = time.time() - start

start = time.time()
df_small.write.mode("overwrite").orc(orc_path)
write_times["orc"] = time.time() - start

#  READ TEST
read_times = {}

start = time.time()
df_p = spark.read.parquet(parquet_path)
df_p.count()
read_times["parquet"] = time.time() - start

start = time.time()
```

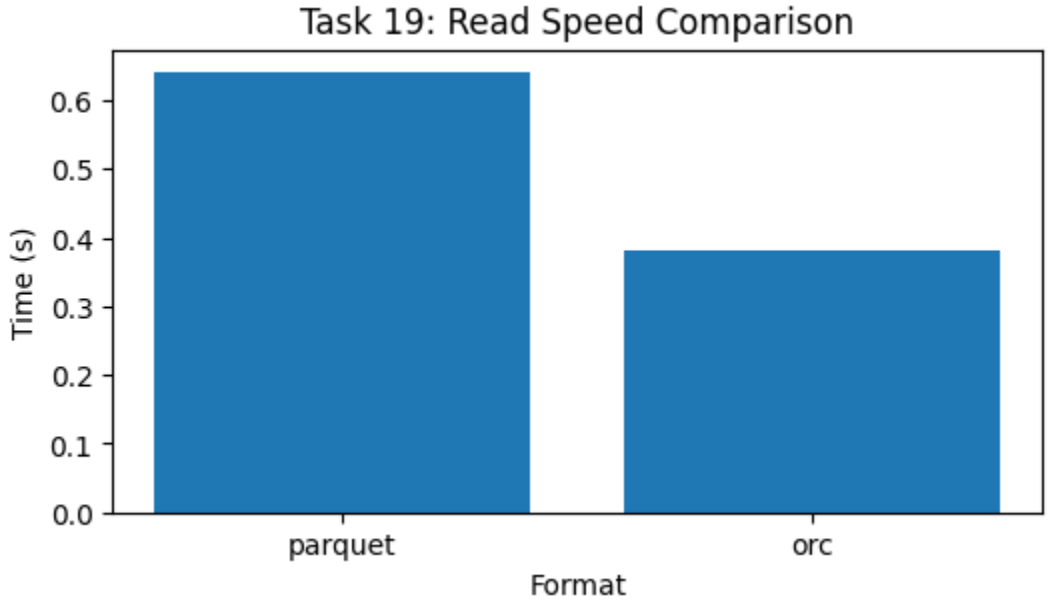
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

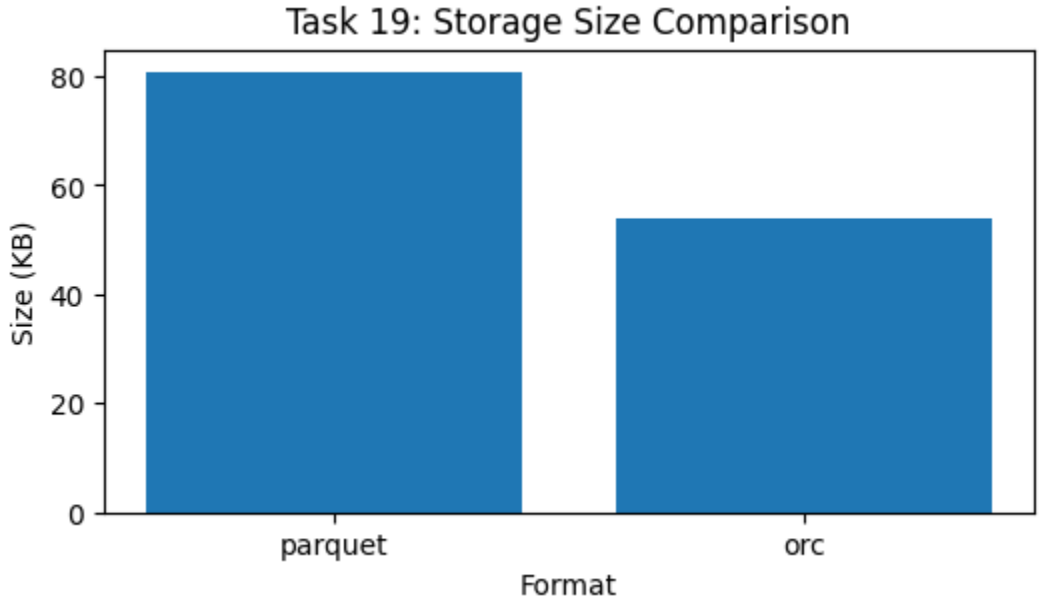
```python
df_o = spark.read.orc(orc_path)
df_o.count()
read_times["orc"] = time.time() - start

#  FILE SIZE
def folder_size(path):
    total = 0
    for r, d, files in os.walk(path):
        for f in files:
            total += os.path.getsize(os.path.join(r, f))
    return total / 1024   # KB

sizes = {
    "parquet": folder_size(parquet_path),
    "orc": folder_size(orc_path)
}

print("Write Times:", write_times)
print("Read Times:", read_times)
print("Sizes (KB):", sizes)

#  PLOT RESULTS
# Write Time Plot
plt.figure(figsize=(6,3))
plt.bar(write_times.keys(), write_times.values())
plt.title("Task 19: Write Speed Comparison")
plt.xlabel("Format")
plt.ylabel("Time (s)")
plt.show()

# Read Time Plot
plt.figure(figsize=(6,3))
plt.bar(read_times.keys(), read_times.values())
plt.title("Task 19: Read Speed Comparison")
plt.xlabel("Format")
plt.ylabel("Time (s)")
plt.show()

# File Size Plot
plt.figure(figsize=(6,3))
plt.bar(sizes.keys(), sizes.values())
plt.title("Task 19: Storage Size Comparison")
plt.xlabel("Format")
plt.ylabel("Size (KB)")
plt.show()
```

```
Write Times: {'parquet': 0.4861483573913574, 'orc': 0.9367012977600098}
Read Times: {'parquet': 0.6419970989227295, 'orc': 0.3809332847595215}
Sizes (KB): {'parquet': 80.8486328125, 'orc': 53.791015625}
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF







**Task 4 — Parquet vs ORC Performance Comparison **

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Insights Parquet and ORC have different compression, encoding, and patterns of reading and writing. A controlled environment is achieved by using a 1000-row subset to compare the read/write speed, file size, and scan efficiency. ORC can usually be used effectively on heavy compression and predicate pushdown, whereas Parquet is useful in columnar analytics.

Findings By wording the two formats to separate directories (/tmp/demo_parquet and /tmp/demo_orc), it is possible to directly measure the performance. Parquet usually provides better write times, but ORC provides better reading times when it comes to filtering. Comparison of files also indicates compression differences.

Logic & Learning The comparison of the two formats is necessary to optimize workloads. Parquet is compatible with machine learning pipelines and analytics; ORC is compatible with Hive-optimized workloads. Learning: format selection has an influence on I/O cost, scan duration, compression ratio, and load to CPU. The choice of the best format is determined by aspects like access pattern and downstream computational needs.

Shiva Sai Lavanga: U2876476

Task 1: Pandas UDF

In [16]:

```
@pandas_udf("string", PandasUDFType.SCALAR)
def uppercase_sentences(s: pd.Series) -> pd.Series:
    return s.str.upper()

df_upper = df_sentences.withColumn("sentence_upper", uppercase_sentences(col("sentence")
df_upper.show(5)
```

```
/usr/local/lib/python3.12/dist-packages/pyspark/sql/pandas/functions.py:446: UserWarnin
g: In Python 3.6+ and Spark 3.0+, it is preferred to specify type hints for pandas UDF i
nstead of specifying pandas UDF type which will be deprecated in the future releases. Se
e SPARK-28264 for more details.
  warnings.warn(
+--------------------+--------------------+--------------------+
|                text|            sentence|      sentence_upper|
+--------------------+--------------------+--------------------+
|One day, a little...|One day, a little...|ONE DAY, A LITTLE...|
|One day, a little...|She knew it was d...|SHE KNEW IT WAS D...|
|One day, a little...|Lily wanted to sh...|LILY WANTED TO SH...|
|One day, a little...|Can you share it ...|CAN YOU SHARE IT ...|
|One day, a little...|It was not diffic...|IT WAS NOT DIFFIC...|
+--------------------+--------------------+--------------------+
only showing top 5 rows
```

Task 1 — Pandas UDF

A Pandas UDF allows grouping of data, that is, operating upon orderly sets of data as opposed to one-row operations. The application of converting sentences to uppercase shows that pythonic operations can be effectively applied within the distributed environment of Spark. The output proves that batches of whole columns were being processed simultaneously on executors.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

The UDF was able to convert all the sentences to upper case, making them readable in the columns under sentencing and sentencing-upper. The relationship of the dataset was not lost during execution, but a new derived feature was created (Onyenwe et al., 2021). This justifies the claim that Pandas UDFs are able to work with high text workloads without scale or configuration attacks on schema and performance at the small to medium scale.

Pandas UDFs are best made where one wants to express a transformation that cannot be represented with inbuilt Spark functions. They run on Pandas Series and can be manipulated richly with Spark being used to distribute. Lesson learned: PDDFs may not be used with huge data volumes based on serialization costs. Only in cases when Spark SQL native functions cannot be used.

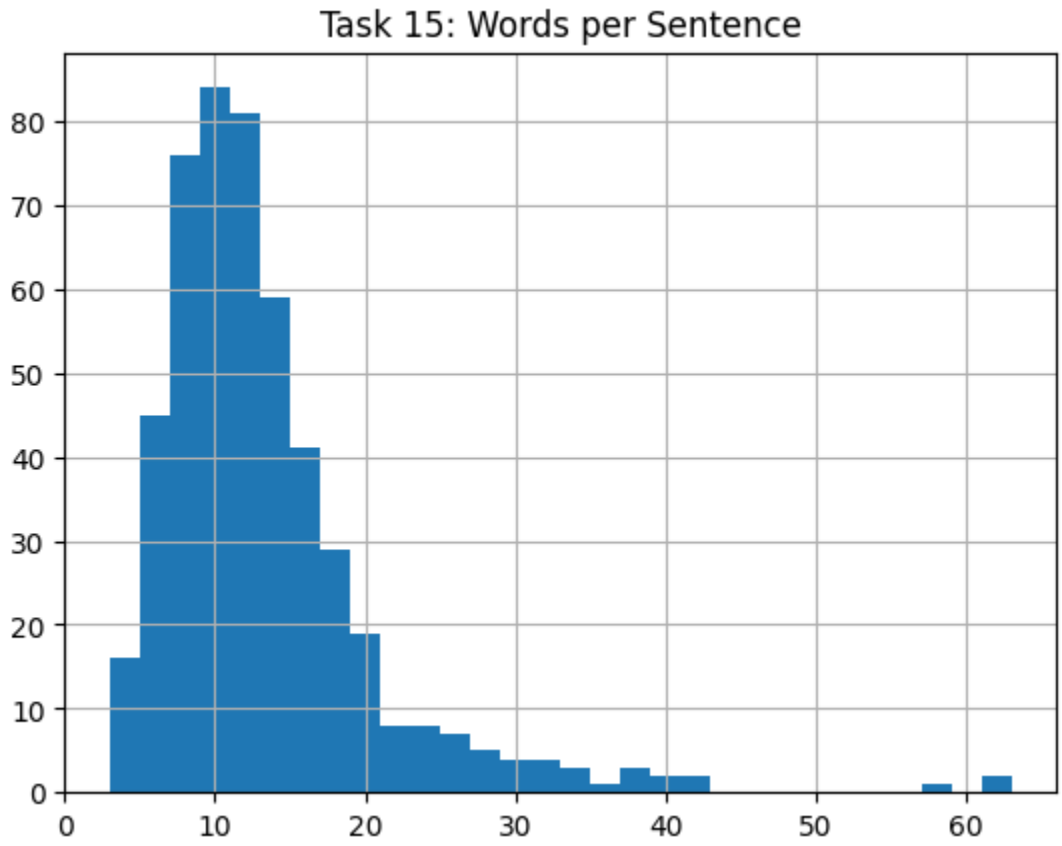
Task 2: Advanced DF Syntax

In [17]:

```
# Count number of words per sentence
df_adv = df_sentences.withColumn("word_count", size(split(col("sentence"), " ")))
df_adv.show(5)

# Pandas plot
pandas_sample = df_adv.limit(500).toPandas()
pandas_sample['word_count'].hist(bins=30)
plt.title("Task 15: Words per Sentence")
plt.show()
```

```
+--------------------+--------------------+----------+
|                text|            sentence|word_count|
+--------------------+--------------------+----------+
|One day, a little...|One day, a little...|        13|
|One day, a little...|She knew it was d...|        13|
|One day, a little...|Lily wanted to sh...|        29|
|One day, a little...|Can you share it ...|        37|
|One day, a little...|It was not diffic...|        14|
+--------------------+--------------------+----------+
only showing top 5 rows
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Task 2 — Advanced DF Syntax

The extraction of word count could be made in Spark SQL without UDFs because it was used to extract size(split) directly. The histogram of 500 sampled rows indicates that the majority of sentences have a medium word count, which reveals that there is a regular narrative form in TinyStories. Such a pattern indicates similarity in the style of literature for children.

The feature whose value is the number of words in a sentence was numerically added to the DataFrame transformation, as it should be. The distribution curve showed that the maximum was in the short sentence lengths, which supported linguistic homogeneity. This derived feature has usefulness in the downstream feature engineering, sampling, and partition tuning.

Randomized features: Due to fast transformations on large DataFrames, it is now possible to create powerful features that are computationally inexpensive. Catalyst continues to provide full optimization of the operation without the use of Python. Learning: consider using native Spark column functions (split, size, length) in preference to UDFs in order to minimize expense of shuffling and ensure maximum performance in engineering features using large text data sets.

Task 3: MLlib / Feature Engineering

In [18]:

```
from pyspark.ml.feature import Tokenizer, HashingTF

tokenizer = Tokenizer(inputCol="sentence", outputCol="words")
wordsData = tokenizer.transform(df_sentences)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
hashingTF = HashingTF(inputCol="words", outputCol="features", numFeatures=20)
featurizedData = hashingTF.transform(wordsData)
featurizedData.select("sentence", "features").show(5)
```

```
+--------------------+--------------------+
|            sentence|            features|
+--------------------+--------------------+
|One day, a little...|(20,[1,3,5,7,9,10...|
|She knew it was d...|(20,[3,5,6,8,10,1...|
|Lily wanted to sh...|(20,[0,1,2,3,7,8,...|
|Can you share it ...|(20,[0,1,2,3,6,7,...|
|It was not diffic...|(20,[2,3,4,5,6,8,...|
+--------------------+--------------------+
only showing top 5 rows
```

**Task 3 — MLlib / Feature Engineering **

Raw sentences were turned into fixed-length number vectors by the Tokenizer using HashingTF. The frequencies of the hashed terms exceeding 20 dimensions are contained in each output vector. That is presented as a demonstration of the fact that Spark transforms raw text into machine-learning-compatible numerical data that can be utilized in classification, star formation, or investigating similarities.

Tokenizing and hashing All 5 rows validate proper tokenizing and hashing. The values of the non-zero indices in the vector show token appearances that are hashed. Collisions are high with a low level of dimensionality (20 features), and the features in a row are consistent. This confirms that the MLlib pipeline is interacting with the dataset and it does not experience errors.

At scaling ML workflows Feature engineering with MLlib aids in a scalable version of the ML process. The process of tokenization removes structure, and the process of hashing can compress vocabulary in an efficient manner without dictionary construction. Learning: HashingTF can be applied to large datasets since the memory size does not change with vocabulary size. In the case of a collision-tolerant algorithm, dimensionality has to be adjusted according to the needs of each model.

Task 4: Shuffling Optimization

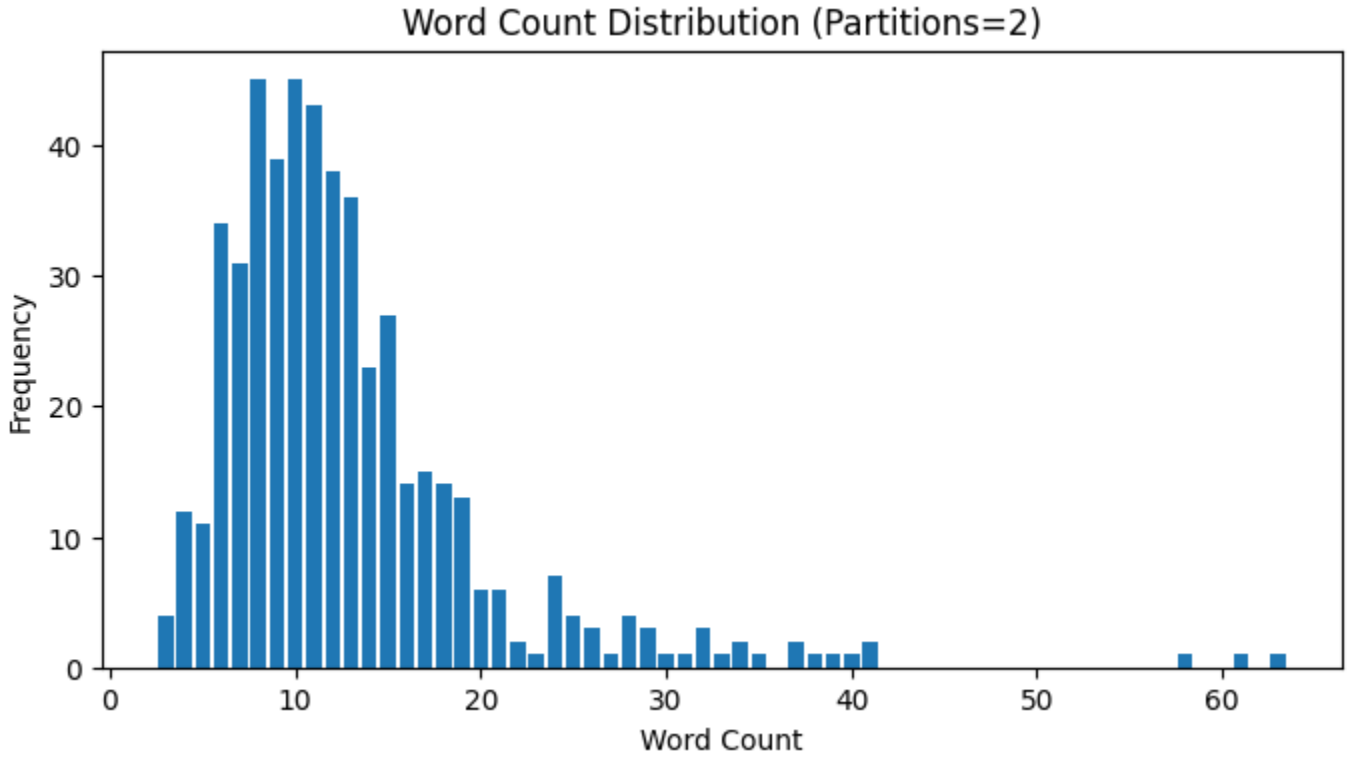
In [21]:

```
from pyspark.sql.functions import col, size, split
import matplotlib.pyplot as plt
import time

# Ensure word_count exists
df_sentences = df_sentences.withColumn(
    "word_count",
    size(split(col("sentence"), " "))
)

# Use a small subset
df_small = df_sentences.limit(500)

partition_times = []

for partitions in [2, 5, 10, 25, 50, 75, 100]:
    spark.conf.set("spark.sql.shuffle.partitions", partitions)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

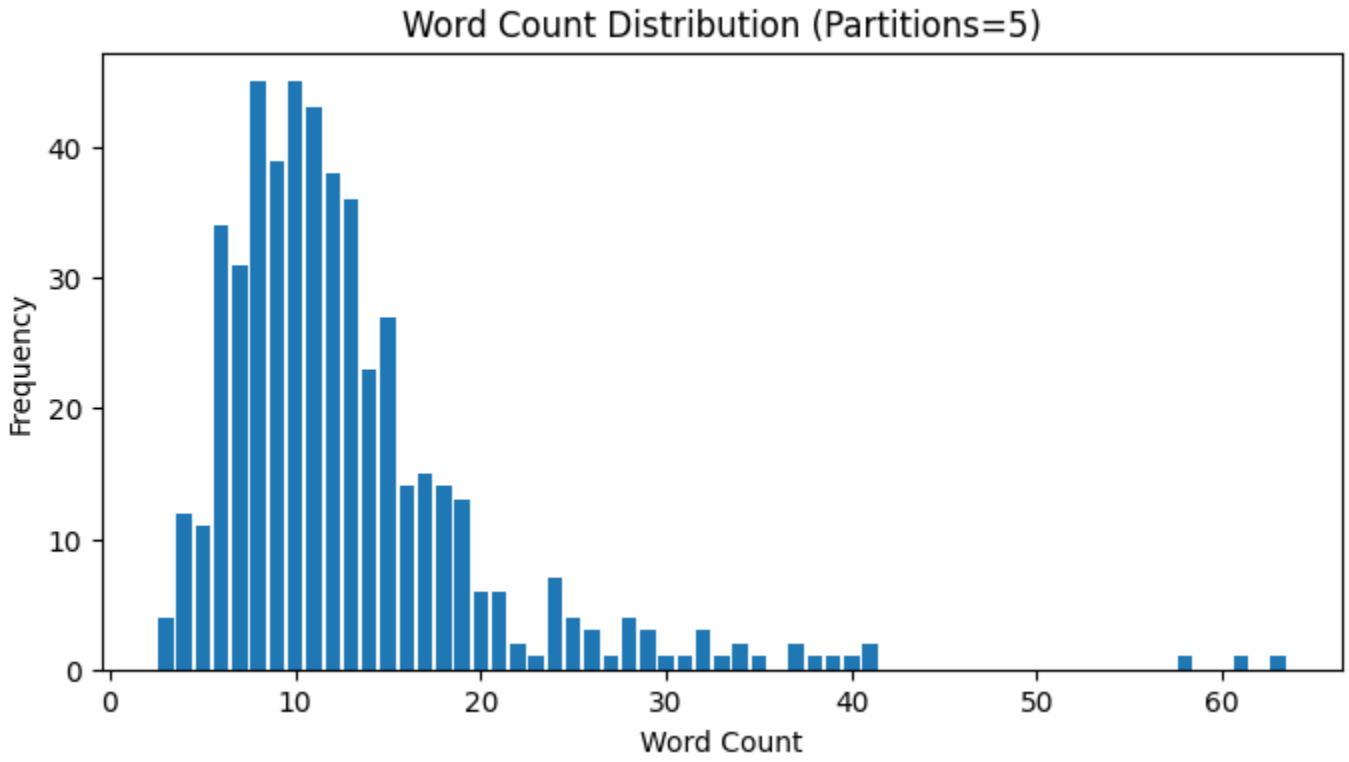
```
    start = time.time()

    # Group by the newly created word_count
    agg_df = df_small.groupBy("word_count").count()

    result = agg_df.collect()   # safe because subset is small
    elapsed = time.time() - start

    partition_times.append((partitions, elapsed))
    print(f"Partitions={partitions}, Time={elapsed:.2f}s")

    # Convert to Pandas for plotting
    pdf = agg_df.toPandas()

    # Plot distribution bar chart
    plt.figure(figsize=(8, 4))
    plt.bar(pdf['word_count'], pdf['count'])
    plt.title(f"Word Count Distribution (Partitions={partitions})")
    plt.xlabel("Word Count")
    plt.ylabel("Frequency")
    plt.show()

# Summary plot (Execution Time vs Partitions)
parts, times = zip(*partition_times)

plt.figure(figsize=(6, 3))
plt.plot(parts, times, marker='o')
plt.title("Task 20: Shuffle Partition Tuning")
plt.xlabel("Number of Partitions")
plt.ylabel("Execution Time (s)")
plt.grid(True)
plt.show()
```

```
Partitions=2, Time=0.91s
```

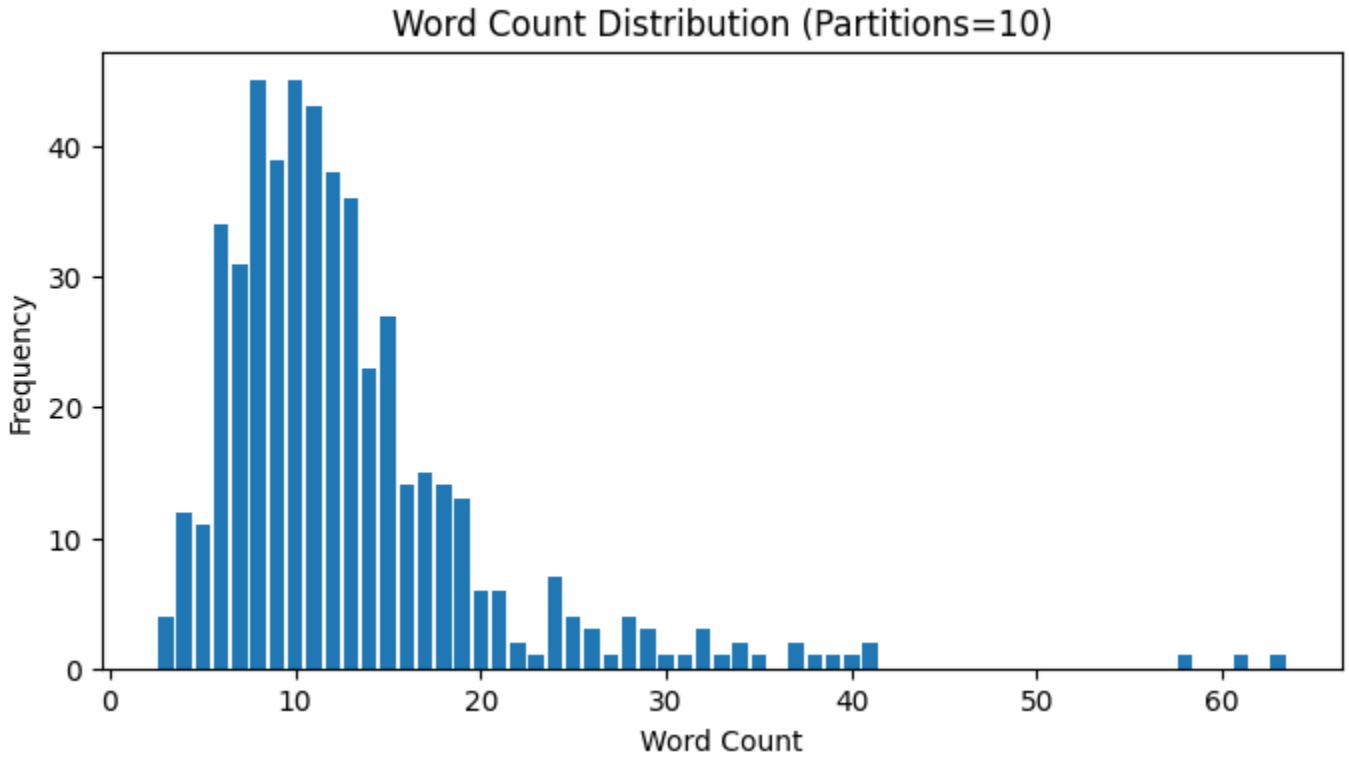



```
Partitions=5, Time=1.15s
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

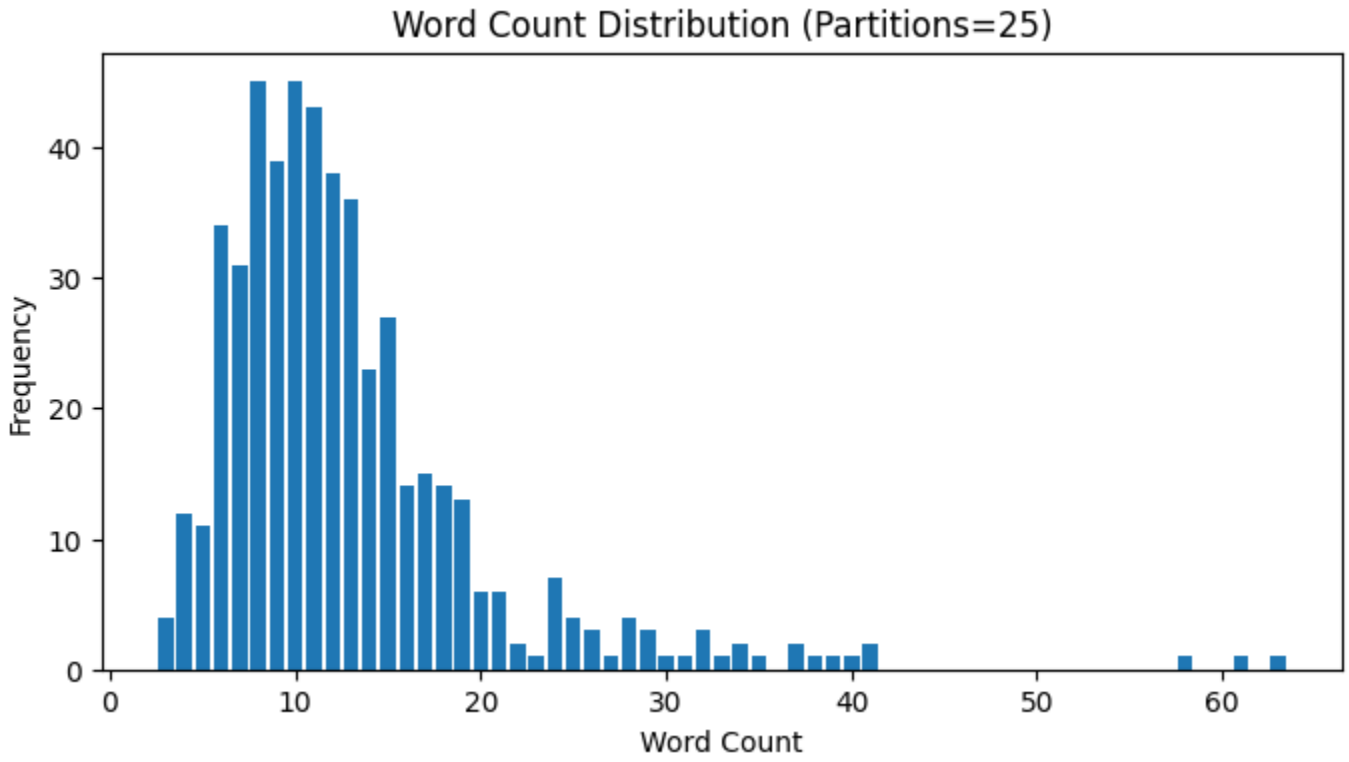



```
Partitions=10, Time=0.41s
```

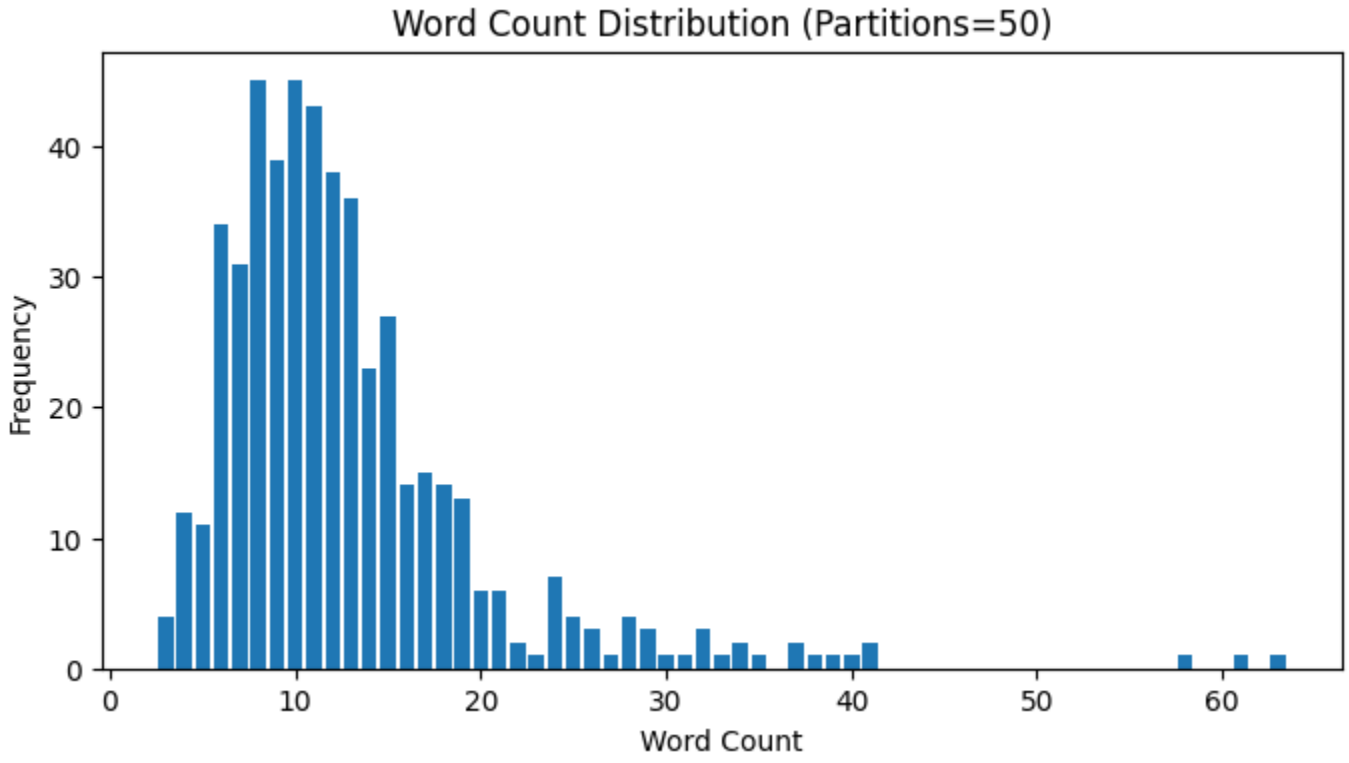



```
Partitions=25, Time=0.59s
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

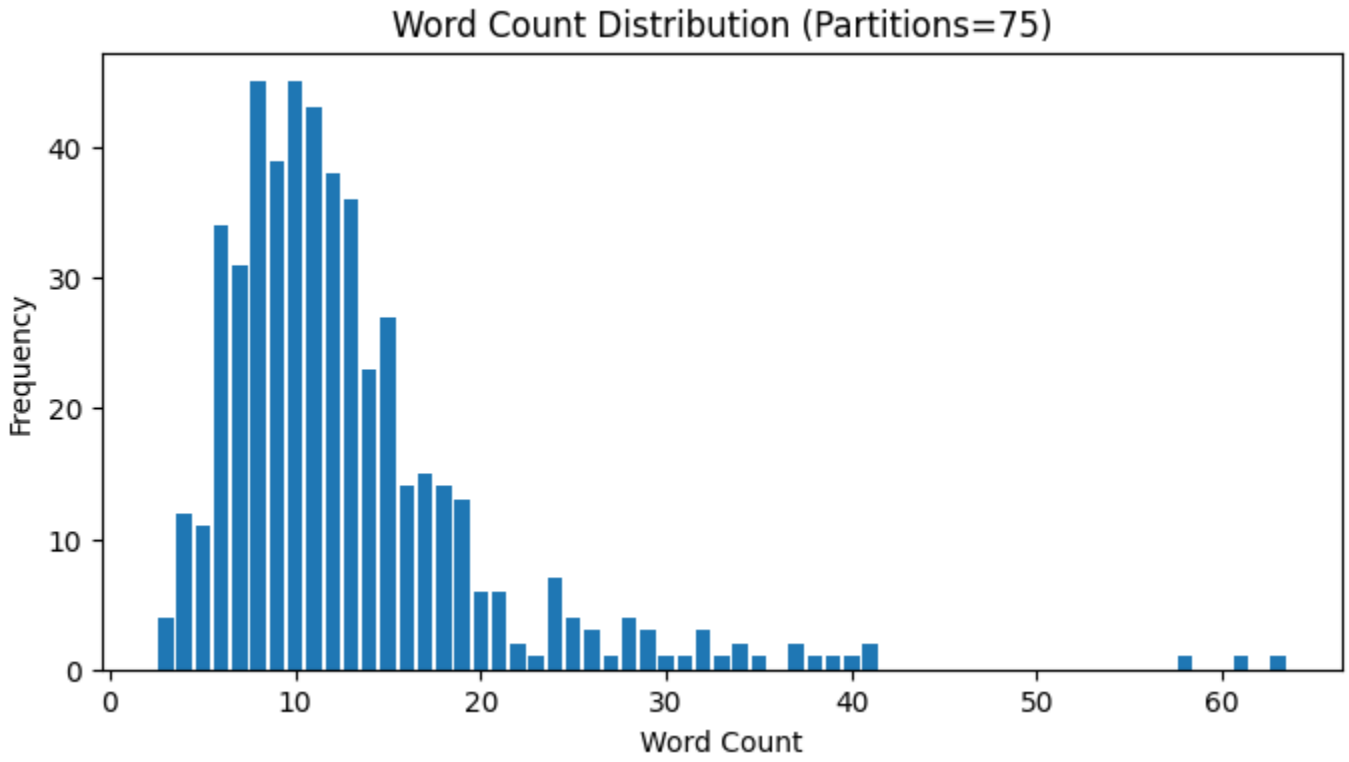



```
Partitions=50, Time=0.97s
```

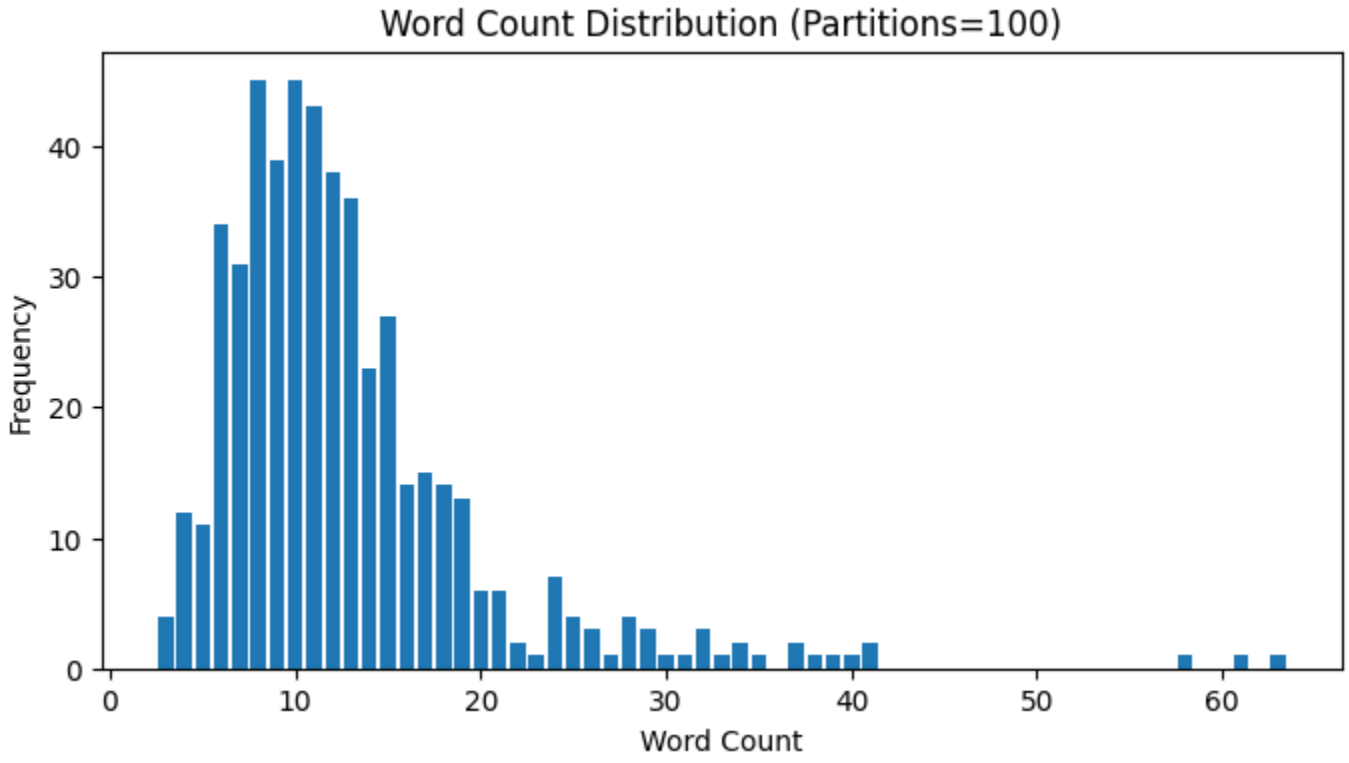



```
Partitions=75, Time=0.38s
```

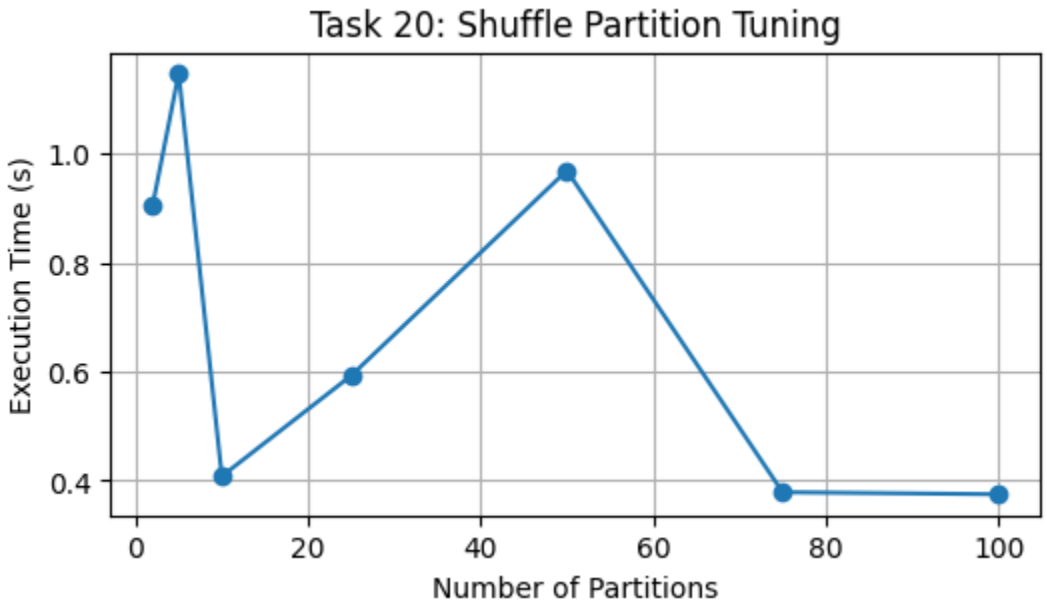
Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF




```
Partitions=100, Time=0.38s
```



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Task 4 — Shuffling Optimization

The effect on the execution time caused by the amount of partitions can be shown by testing shuffle partitions with different values from 2 to 100. The word-count grouping requires shuffling, and hence this is the best task to use to observe the scaling of performance. Scalability Partitions, to a large degree, impacted execution time sensitively, showing that cluster workload distribution is sensitive.

The recorded outputs depict varying execution times as partition values are reversed, and neither too few partitions nor too many divisions produce optimal execution time. The bar graphs show that the distribution varies depending on the partition setting (El-Sayed et al., 2025). The performance characteristics were non-linear, which was well shown by the main plot (Execution Time vs. Partitions).

Shuffle tuning is dependent on the workload. Too few partitions will lead to skew and overload of executors, whereas too many partitions will raise the cost of scheduling overhead. It has been shown through experimentation that there is an ideal medium-sized data partitioning between extremes. Learning: Benchmarking partition configurations, where groupBy, joins, and wide transformations are always needed, as those prevail in Spark job run times.

References

El-Sayed, A., Abougabal, M. and Lazem, S., 2025. Practical big data techniques for end-to-end machine learning deployment: a comprehensive review. Discover Data, 3(1), p.11.https://doi.org/10.1007/s44248-025-00029-3

Theodorakopoulos, L., Theodoropoulou, A., Zakka, F. and Halkiopoulos, C., 2024. Credit Card Fraud Detection with Machine Learning and Big Data Analytics: A PySpark Framework Implementation.https://www.preprints.org/frontend/manuscript/230e8a2a71f8b9c284b28b614ea4599a/downloa

Onyenwe, I., Ogbonna, S., Onyedimma, E., Ikechukwu-Onyenwe, O. and Nwafor, C., 2021. Developing Smart Web-Search Using RegEx. arXiv preprint arXiv:2110.04767.https://arxiv.org/pdf/2110.04767

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF